

CANDEX HANDBOOK

API / Contracts

The Candex REST API reference — conventions, the response envelope, status codes, headers, authentication, and the endpoints per resource.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

API / Contracts

This is the contract reference for `candex-api`. It builds on Back-end (how the API is built) and lists the endpoints per resource. The **live, exhaustive contract** — request/response schemas, examples, and try-it-out — is **Swagger UI** at `/swagger-ui.html`; this page is the stable map around it.

Conventions

- **Envelope** — every response is a `Result<T>` in Java, serialized as the bare payload on success or the bare `Error` on failure (see Back-end).
- **Status codes** — come from typed exceptions (`404` / `400` / `409` / `422`…); see Back-end error handling.
- **Headers** — list responses carry `X-Total-Count`; creations return `201` with a `Location` header.
- **Authentication** — send a Firebase ID token as `Authorization: Bearer <token>`.
- **Access** — per endpoint, in the tables below: `USER_OR_ADMIN` = any authenticated user, `ADMIN` = admin only, `PUBLIC` = no auth, **own** = the authenticated user may only act on their own records.
- **Base path** — resources are served under the API base (for example `/api/drinks`).

Calling the API

A read sends the Firebase ID token and gets the bare DTO back:

```
curl https://<api-host>/api/users/firebase/4q8ESFox1Scd8WMT438M5UlZGMy1 \
  -H "Authorization: Bearer $FIREBASE_ID_TOKEN"
```

```
{
  "id": 42,
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "emailVerified": true,
  "joinedInVersion": "1.1.10"
}
```

Registration (`POST /users`) is what the app sends right after creating the Firebase account. `id`, `emailVerified` and `joinedInVersion` are set by the API, never by the caller:

```
{
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "birthDate": "2001-07-15"
}
```

It answers `201 Created`, a `Location: /users/42` header, and the created user. Unlocking a drink is the smallest write in the API:

```
{
  "userId": 42,
  "drinkId": 7
}
```

It answers with the stored unlock and its server-side timestamp:

```
{
  "userId": 42,
  "drinkId": 7,
  "unlockedAt": "2026-11-12T20:52:52"
}
```


Every failure has the same shape, whatever the endpoint — here, registering a username that is taken:


```
{
  "name": "UniqueConstraintViolationException",
  "message": "A UserEntity with username = 'therealinvestigator' already exists.",
  "status": 409,
  "timestamp": "2026-11-12 20:52:52"
}
```


The client can branch on `status` for the category of error and on `name` for the exact case.

Catalogue

Drinks — `/drinks`

Method	Path	Access	Purpose
GET	/drinks	USER_OR_ADMIN	List drinks
GET	/drinks/{id}	USER_OR_ADMIN	A single drink
POST	/drinks	ADMIN	Create a drink
POST	/drinks/import	ADMIN	Bulk import drinks
POST	/drinks/import/diff	ADMIN	Preview an import against the current catalogue
POST	/drinks/import/sync	ADMIN	Apply an import
PUT	/drinks/{id}/shift	ADMIN	Move the drink's World Dex position
PUT	/drinks/{id}/swap	ADMIN	Swap two World Dex positions
DELETE	/drinks/{id}	ADMIN	Delete a drink

Cans — /cans

Method	Path	Access
GET	/cans • /cans/{id}	USER_OR_ADMIN
POST	/cans	ADMIN
DELETE	/cans/{id}	ADMIN

Reference data (reads)

Catalogue reference resources expose list and by-id reads: **Brands** /brands, **Flavors** /flavors and **Flavor categories** /flavor-categories, **Colors** /colors (+ /colors/extract), **Drink categories** /drink-categories, **Can formats** /can-formats, and **App versions** /app-versions (+ /app-versions/current).

World Dex — `/world-dex`

Method	Path	Access	Purpose
GET	/world-dex/drinks/{id}/history	USER_OR_ADMIN	A drink's position history
GET	/world-dex/drinks/{id}/stats	USER_OR_ADMIN	A drink's Dex stats
GET	/world-dex/snapshot	USER_OR_ADMIN	Current Dex snapshot
GET	/world-dex/movers	USER_OR_ADMIN	Recent movers
GET	/world-dex/positions/{position}/occupants	USER_OR_ADMIN	Who sits at a position

Users & collection

Users — `/users`

Method	Path	Access	Purpose
GET	/users	ADMIN	List users
GET	/users/{id}	own	A user
GET	/users/firebase/{firebaseUid}	own	A user by Firebase UID
GET	/users/{id}/drinks	own	A user's unlocked drinks
GET	/users/exists	PUBLIC	Existence check
POST	/users	PUBLIC	Register
PUT • PATCH	/users/{id}	own	Update a user
DELETE	/users/{id}	own	Delete a user

Unlocked drinks — `/user-unlocked-drinks`

Method	Path	Access	Purpose
GET	/user-unlocked-drinks/{userId}	own	A user's unlocks
POST	/user-unlocked-drinks	own	Unlock a drink
DELETE	/user-unlocked-drinks/{userId}/{drinkId}	own	Remove an unlock

Admin & operations

- **SQL console** — `POST /admin/...` (ADMIN): run a read-only query through the console role (see Database & Model).
- **Reports** — `POST /reports`: generate a report.

The bulk-import flow

Importing a batch of drinks is a three-step, admin-only pipeline:

1. `POST /drinks/import` — submit the payload.
2. `POST /drinks/import/diff` — preview the changes against the current catalogue.
3. `POST /drinks/import/sync` — apply them.

Where to go next

- How the API is built → Back-end
- Live schemas & try-it-out → **Swagger UI** (`/swagger-ui.html`)